1 Port Expander LCD

O módulo Port Expander LCD (PELCD) atua como interface entre a unidade de processamento central (Control) e o LCD. A sua função principal é receber, através do seu bloco interno Serial Receiver, tramas de 10 bits transmitidas de forma serial bit a bit e convertê-las para o formato paralelo necessário para controlar os pinos de dados e de controlo do LCD.

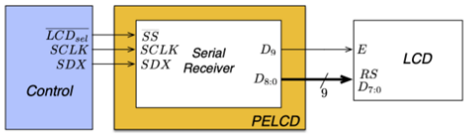
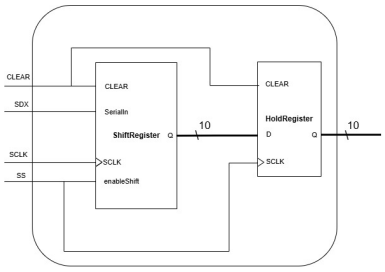


Figura 1: *Diagrama de blocos LCD*

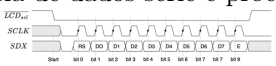
2 Serial Receiver

O bloco *Serial Receiver* apresentado na figura 2a, é um sistema de receção de dados em série composto por dois componentes principais. O *Shift Register* e o *Hold Register*. O funcionamento baseia-se na entrada de dados série (SDX) que são deslocados para o *Shift Register* em função do sinal do clock ($SCLK$), desde que o sinal de seleção (SS) esteja ativo .



(a) Serial Receiver

Uma vez concluída a receção dos 10 bits, a transição do sinal que funciona aqui como um sinal de controlo de carga por flanco, transfere o conteúdo paralelo do registo de deslocamento para o *Hold Register*, garantindo que os dados fiquem estáveis na saída do sistema enquanto uma nova sequência de dados série é processada.



(b) Diagrama temporal

Figura 2: Bloco *Key Decode*

3 Controlador

O emissor, realizado em software, quando pretende enviar uma trama para o módulo *PELCD* promove uma condição de início de trama (Start), que corresponde a uma transição descendente na linha ($\overline{LCDsel}$). Após a condição de início, o módulo PELCD armazena os bits de dados da trama nas transições ascendentes do sinal $SCLK$.

Devido às especificações de temporização do controlador do próprio LCD, é imperativo garantir que os sinais de dados e de controlo estejam estáveis antes e depois da ativação do sinal de Enable (E).

Na arquitetura implementada, em que a comunicação é feita através do módulo *Port Expander LCD* ($PELCD$), este requisito obriga a um protocolo de envio duplo por parte do emissor em software. Para efetivar a escrita de um byte (seja um comando ou um dado), o software deve enviar duas tramas consecutivas de **10 bits** para o *Serial Receiver*:

- **Primeira Trama**: Define o bit RS (bit 0) e os dados (bits 1 a 8), configurando o bit 9 (Enable) a 1.
- **Segunda Trama**: Mantém os mesmos valores para RS e dados, mas altera o bit 9 (Enable) para 0.

Esta sequência garante que o LCD detete a transição descendente (*falling edge*) necessária no sinal de Enable para registar a informação presente nos seus pinos de entrada. A utilização de um *Hold Register* no bloco *Serial Receiver* é fundamental neste processo, pois permite que os sinais permaneçam estáveis e disponíveis na saída do periférico durante todo o tempo em que as tramas série estão a ser processadas, assegurando a integridade da comunicação

3.1 LCD

Para a programação do visor de cristais líquidos (Liquid Crystal Display - LCD), foi implementada

a sequência de comandos padrão definida pelo controlador HD44780 para garantir a sua correta inicialização e configuração. Este processo é realizado através do envio de tramas de 10 bits via protocolo série para o módulo PELCD.

O procedimento inicia-se com uma fase de sincronização, na qual é enviado o comando 0x30 (00110000) com o bit RS a 0 (modo comando) três vezes consecutivas, respeitando os intervalos de tempo obrigatórios entre cada envio para estabilização do hardware.

Após esta fase, são transmitidos os seguintes comandos de configuração:

- **Function Set** (00111000): Configura a interface e define que o visor operará com 2 linhas de texto e que cada caráter será desenhado numa matriz de 5x8 pontos.

- **Display ON/OFF** (00001111): Este comando é responsável por ligar o ecrã (D=1), ativar a visualização do cursor (C=1) e definir que o mesmo deve permanecer em modo intermitente/piscar (B=1).

- **Entry Mode** (00000110): Define o modo de introdução de dados, estabelecendo que o endereço da memória DDRAM deve ser incrementado automaticamente após a escrita de cada caráter e que o ecrã deve permanecer estático (sem deslocamento da imagem).

A execução correta desta sequência, respeitando os tempos de espera (wait times) indicados no datasheet, é fundamental para que o LCD saia do seu estado de repouso e fique pronto para receber os dados a visualizar na DDRAM

4 Conclusões

O desenvolvimento do módulo PELCD permitiu validar a eficácia da comunicação série-paralelo na otimização de recursos de hardware, reduzindo drasticamente o número de linhas de I/O necessárias para controlar o LCD. A implementação da arquitetura baseada em Shift e Hold Registers revelou-se fundamental para garantir a estabilidade dos sinais exigida pelo controlador do LCD.

Embora o protocolo síncrono e a lógica de controlo tenham sido validados com sucesso, a transição do software para o hardware apresentou alguns desafios significativos, concretamente na visualização de caracteres no display. Identificou-se que a principal causa de erro residia na ordenação dos bits durante a transmissão série. O software enviava s bits na ordem da direita para a esquerda , enquanto o hardware esperava uma sequência da esquerda para a direita. Esta inversão da trama impossibilitava a descodificação correta dos comandos pelo controlador do LCD. E só quando corrigimos a rotina de envio para garantir a ordem correta de bits é que o sistema passou a operar conforme esperado.

Figura 3: *Primeira entrega funcional*

A VHDL

A.1 Serial Receiver

```
library ieee;
use ieee.std_logic_1164.all;

ENTITY SerialReceiver IS
        PORT(
                SCLK, SDX, SS, CLEAR:         IN std_logic;
                Q:                                                      OUT std_logic_vector(9 downto 0)
        );
END SerialReceiver;

ARCHITECTURE Behaviour OF SerialReceiver IS

        component ShiftRegister
                PORT(
                        SCLK, enableShift, SerialIn, CLEAR:         IN std_logic;
                        Q:                                                                      OUT std_logic_vector(9 do
                );
        end component;

        component HoldRegister
                PORT(
                        SCLK, CLEAR:         IN std_logic;
                        D:                                      IN std_logic_vector(9 downto 0);
                        Q:                                      OUT std_logic_vector(9 downto 0)
                );
        end component;

        signal shiftRegisterOut: std_logic_vector(9 downto 0);
        signal enableShift, holdShiftCLK: std_logic;

BEGIN

        enableShift                <= SS;

        shiftRegister1: ShiftRegister port map(
                SCLK                                    => SCLK,
                enableShift         => enableShift,
                SerialIn                 => SDX,
                CLEAR                                   => CLEAR,
                Q                                            => shiftRegisterOut
        );

        hold: HoldRegister port map(
                SCLK                                    => SS,
                D                                            => shiftRegisterOut,
                CLEAR                                   => CLEAR,
                Q                                            => Q
        );

END Behaviour;
```

A.2 ShiftRegister

```
library ieee;
use ieee.std_logic_1164.all;

ENTITY ShiftRegister IS
        PORT(
                SCLK, enableShift, SerialIn, CLEAR:         IN std_logic;
                Q:                                                                                                  OUT std_l
        );
END ShiftRegister;

ARCHITECTURE Behaviour OF ShiftRegister IS

        component RegistryL4
                PORT(
                        D: IN std_logic_vector (3 downto 0);
                        clk_in, CE, CLEAR: IN std_logic;
                        Q: OUT std_logic_vector (3 downto 0)
                );
        end component;

        component RegistryL2
                PORT(
                        D: IN std_logic_vector (1 downto 0);
                        clk_in, CE, CLEAR: IN std_logic;
                        Q: OUT std_logic_vector (1 downto 0)
                );
        end component;

        signal D, D_NEXT: std_logic_vector(9 downto 0);
        signal registerEnable: std_logic;


BEGIN

        registerEnable <= NOT enableShift;

        registry1: RegistryL4 port map(
                clk_in => SCLK,
                CLEAR  => CLEAR,
                D => D_NEXT(3 downto 0),
                CE => registerEnable,
                Q => D(3 downto 0)
        );

        registry2: RegistryL4 port map(
                clk_in => SCLK,
                CLEAR  => CLEAR,
                D => D_NEXT(7 downto 4),
                CE => registerEnable,
                Q => D(7 downto 4)
        );

        registry3: RegistryL2 port map(
                clk_in => SCLK,
                CLEAR  => CLEAR,
                D => D_NEXT(9 downto 8),
                CE => registerEnable,
                Q => D(9 downto 8)
        );

        Q <= D;

        process(D_NEXT, D, SerialIn)
        begin
```



```
    D_NEXT(9 downto 1) <= D(8 downto 0);
    D_NEXT(0) <= SerialIn;

        end process;
```

A.3 HoldRegister

```
library ieee;
use ieee.std_logic_1164.all;

ENTITY HoldRegister IS
        PORT(
                SCLK, CLEAR        :          IN std_logic;
                D:                                          IN std_logic_vector(9 downto 0);
                Q:                                          OUT std_logic_vector(9 downto 0)
        );
END HoldRegister;

ARCHITECTURE Behaviour OF HoldRegister IS

        component RegistryL4
                PORT(
                        D: IN std_logic_vector (3 downto 0);
                        clk_in, CE, CLEAR: IN std_logic;
                        Q: OUT std_logic_vector (3 downto 0)
                );
        end component;

        component RegistryL2
                PORT(
                        D: IN std_logic_vector (1 downto 0);
                        clk_in, CE, CLEAR: IN std_logic;
                        Q: OUT std_logic_vector (1 downto 0)
                );
        end component;

BEGIN

        registry1: RegistryL4 port map(
                clk_in => SCLK,
                CLEAR  => CLEAR,
                D => D(3 downto 0),
                CE => '1',
                Q => Q(3 downto 0)
        );

        registry2: RegistryL4 port map(
                clk_in => SCLK,
                CLEAR  => CLEAR,
                D => D(7 downto 4),
                CE => '1',
                Q => Q(7 downto 4)
        );

        registry3: RegistryL2 port map(
                clk_in => SCLK,
                CLEAR  => CLEAR,
                D => D(9 downto 8),
                CE => '1',
                Q => Q(9 downto 8)
        );


END Behaviour;
```

B *Kotlin*

B.1 LCD

Algoritmo 1: LCD

```
package org.example

import isel.leic.simul.module.LCD
import isel.leic.utils.Time

// Escreve no LCD usando a interface a 8 bits.
object LCD {

    // Dimens o do display.
    const val LINES = 2
    const val COLS = 16

    // Escreve um byte de comando/dados no LCD em s rie
    private fun writeByteSerial(rs: Boolean, data: Int) {
        val rsBit = if (rs) 1 else 0

        val extendedData = Integer.toBinaryString(data).padStart(8, '0')

        var dataFullEnabled = "1${extendedData}${rsBit}".toInt(2)
        SerialEmitter.send(addr = SerialEmitter.Peripheral.LCD, dataFullEnabled)

        dataFullEnabled = "0${extendedData}${rsBit}".toInt(2)

        SerialEmitter.send(addr = SerialEmitter.Peripheral.LCD, dataFullEnabled)
    }

    // Escreve um byte de comando/dados no LCD
    private fun writeByte(rs: Boolean, data: Int) {

    }

    // Escreve um comando no LCD
    private fun writeCMD(data: Int) {
        writeByteSerial(rs = false, data)
    }

    // Escreve um dado no LCD
    private fun writeDATA(data: Int) {
        writeByteSerial(rs = true, data)
    }

    // Envia a sequ ncia de inicia  o para comunica  o a 8 bits.
    fun init() {
        SerialEmitter.init()
        println("INICIALIZACAO␣DO␣LCD\n\n")
        Time.sleep(15)
        writeCMD(0b00110000)
        Time.sleep(5)
        writeCMD(0b00110000)
        Time.sleep(1)
        writeCMD(0b00110000)
        Time.sleep(10)
        println("FIM␣DA␣INICIALIZACAO␣DO␣LCD\n\n")
        println("CONFIGS␣DO␣LCD\n\n")
        writeCMD(0b00111000) //FUNCTION SET
        writeCMD(0b00001111) //DISPLAY OFF
        writeCMD(0b00000110) //Define o ENTRY MODE para incrementar automaticamente.
        Time.sleep(100)
        clear()
        println("FIM␣DAS␣CONFIGS␣DO␣LCD\n\n")

        write("Welcome!")
    }

    // Escreve um caracter na posi  o corrente.
    fun write(c: Char) {
```

```
        writeDATA(c.code)
    }

    // Escreve uma string na posi   o corrente.
    fun write(text: String) {
        text.map { write(c = it) }
    }

    // Envia comando para posicionar cursor ('line': 0..LINES-1, 'column': 0..COLS-1)
    fun cursor(line: Int, column: Int) {
        if (line in 0..<LINES && column in 0..<COLS) {
            val lineBits = Integer.toBinaryString(line).padStart(2, '0')
            val columnBits = Integer.toBinaryString(column).padStart(4, '0')

            val cursorCommand = "1$lineBits$columnBits".toInt(2) //USES DDRAM

            writeCMD(data = cursorCommand)
        }
    }

    // Envia comando para limpar o ecr   e posicionar o cursor em (0,0)
    fun clear() {
        writeCMD(data = 0b00000001)  // Clears Display
        Time.sleep(2)
        writeCMD(data = 0b00000010)  // Return Home
        Time.sleep(2)
    }
}
```

B.2 SerialEmitter

Algoritmo 2: SerialEmitter

```
package org.example

import isel.leic.utils.Time

// Envia tramas para os diferentes m dulos Serial Receiver .
object SerialEmitter {
    var busy = false

    enum class Peripheral { LCD, TICKET }

    // Inicia a classe
    fun init() {
        busy = false
    }

    // Envia um a trama para o Serial Receiver
    // identificado o perif rico de destino em 'addr',
    // os bits de dados em 'data'
    // e em 'size' o n mero de bits a enviar.
    fun send(addr: Peripheral, data: Int) {
        if (isBusy()) return
        if (addr == Peripheral.LCD) {
            busy = true
            //println("\nNova Trama")


            /*
            * O BIT (0) VAI SER USADO PARA ENVIAR O COMANDO (SDX)
            * O BIT (1) VAI SER O CLOCK DO SERIAL RECIEVER (SCKL)
            * O BIT (2) VAI SER O ENABLE DO PROCESSO [ACTIVE LOW].
            *
            * */

            println("DADOS␣A␣ENVIAR:␣${Integer.toBinaryString(data).padStart(10,␣'0').reversed()}")


            HAL.clrBits(mask = 0b00000111) // LIMPA OS 3 BITS QUE VAO SER USADOS
            //------000

            //
```

```
            //SE FOR PARA EXECUTAR NA PLACA, N O PODEMOS INVERTER OS BITS. SE FOR PARA EXECUTAR NO SIMUL, TEM
            //
            Integer.toBinaryString(data).padStart(10, '0')
                .reversed()
                .mapIndexed { index, it ->

                    if (it.digitToInt() == 1) {
                        HAL.setBits(mask = 0b00000001) //Fica o ultimo bit ON
                    } else {
                        HAL.clrBits(mask = 0b00000001) //Fica o ultimo bit OFF
                    }

                    HAL.setBits(mask = 0b00000010)
                    /*println("index: $index -> val: $it")*/
                    Time.sleep(10)
                    HAL.clrBits(mask = 0b00000010)
                    Time.sleep(10)
                }
            HAL.clrBits(0b00000111)
            HAL.setBits(0b00000100)
            //println("Fim do Envio de dados \n")
            busy = false
        }
    }

    // Retorna informa  o se o perif rico est  ocupado
    fun isBusy(): Boolean {
        return busy
    }
}
```